

SUPER KB-ZIP 8.4.1

STATUS: CONSTITUTIONAL (LOCKED)

SCOPE: CONTAINER NORMALIZATION ONLY

MODULE: MODULE 1 — CONTAINER NORMALIZATION

AUTHORITY: DERIVED FROM MODULE 0 GOVERNANCE

UPSTREAM OWNER: MC (Phase 0 + Canonical Classification + Routing)

ENFORCEMENT COMPANION: MS (START-FIRST + Structural Lock)

TRACE COMPANION: MT (Visibility Only)

MUTATION RIGHTS: NONE

RI / MRS RIGHTS: NONE

COS RIGHTS: NONE

PURPOSE (LOCKED)

ZIP performs deterministic normalization of compressed container artifacts.

ZIP MUST:

1. Detect container eligibility (as routed by MC).
2. Decompress safely under hard limits.
3. Produce a deterministic manifest.
4. Return extracted artifacts with required provenance metadata to the governed caller for downstream handoff.

ZIP is plumbing.

ZIP is not intelligence.

ZIP does not interpret content.

ZIP does not speak conversationally.

PHASE 0 RESPONSIBILITY (LOCKED)

ZIP is container extraction only.

Phase 0 readiness, heartbeat emission, structural gating, closure-only release enforcement, non-yield ownership, retries, and user-visible containment are governed by MS under MC routing.

If ZIP is invoked before Phase 0 is INGEST_READY, ZIP MUST return a non-user-visible deterministic failure to the caller (MS) and MUST NOT emit any user-facing text.

END PHASE 0 RESPONSIBILITY (LOCKED)

SUPPORTED CONTAINERS (LOCKED)

ZIP MUST support at minimum:

- .zip

DOUBLE-EXTENSION NORMALIZATION (LOCKED)

If inbound filename ends with repeated “.zip” extensions (e.g., “.zip.zip”):

- ZIP MUST normalize container_type to “zip”
- ZIP MUST treat the container as a supported .zip type
- This normalization affects container_type only
- All safety limits and proof binding remain unchanged

Failure to normalize repeated extensions constitutes:

terminal_state: ZIP_FAILURE failure_class:

UNSUPPORTED_CONTAINER_TYPE

Optional additional formats MAY be supported only if explicitly enumerated in configuration.

If container type is unsupported:

terminal_state: ZIP_FAILURE

failure_class: UNSUPPORTED_CONTAINER_TYPE

ZIP MUST NOT attempt ambiguous decompression.

CONTAINER-SET MODE COMPATIBILITY (LOCKED)

ZIP MUST accept the `container_set_mode` committed upstream by MC/MS for the active governed turn.

Admissible values:

- `SINGLE_CONTAINER`
- `ZIP_SEGMENTED_ARCHIVE_SET`
- `ZIP_CONTAINER_BATCH`
- `ZIP_MULTI_INDEPENDENT`

ZIP MUST NOT infer segmented-archive status solely from:

- shared filename prefixes
- timestamps
- numbered `.zip` suffixes
- missing numbered sequence members

If upstream `container_set_mode` is:

- `ZIP_CONTAINER_BATCH`

OR

- `ZIP_MULTI_INDEPENDENT`

ZIP MUST treat each routed ZIP container as independently extraction-eligible.

ZIP MUST NOT:

- require a complete numbered sequence
- wait for missing ZIP files
- attempt segmented reconstruction
- emit or return any bundle field implying “remaining archive parts required”

If upstream container_set_mode is:

- ZIP_SEGMENTED_ARCHIVE_SET

then segmented reconstruction eligibility and waiting posture remain governed upstream under routed handoff contracts.

ZIP itself remains extraction-only and MUST NOT create user-facing waiting semantics.

If ZIP observes internally inconsistent container-set semantics for the same governed turn, ZIP MUST terminate deterministically:

terminal_state: ZIP_FAILURE failure_class:

ZIP_CONTAINER_SET_MODE_CONFLICT

END CONTAINER-SET MODE COMPATIBILITY (LOCKED)

GOOGLE TAKEOUT NUMBERED ZIP INDEPENDENCE (LOCKED)

Purpose:

Prevent Google Takeout numbered ZIP containers from being misinterpreted as segmented multi-volume archives.

Rule:

If upstream classification or routed container evidence indicates a Google Takeout export family, and each routed container is itself a complete .zip file whose filename matches the pattern:

- takeout--<family_or_sequence>-001.zip

then ZIP MUST treat each such container as an independently extraction-eligible ZIP container.

For this Google Takeout pattern, the following are prohibited as gating conditions:

- first_segment_present == TRUE as a prerequisite for extraction
- complete_archive_set as a prerequisite for extraction
- required_parts enumeration as a prerequisite for extraction
- INCOMPLETE_ARCHIVE_SET as a terminal or blocking posture
- EXTRACTION_DEFERRED solely because lower-numbered family/sequence members are absent
- any interpretation that the <family_or_sequence> number implies segmented-archive part order

Clarifier:

For Google Takeout numbered ZIPs, the trailing -001.zip identifies an individual ZIP file and does NOT, by itself, imply that earlier or later <family_or_sequence> numbers are required to make the current ZIP extractable.

Accordingly, when such files are routed under:

- ZIP_CONTAINER_BATCH

OR

- ZIP_MULTI_INDEPENDENT

ZIP MUST:

- extract each routed container independently
- finalize manifest and proof independently per container
- return extracted artifacts independently with provenance for downstream handoff
- never emit or return:
- structural_status = INCOMPLETE_ARCHIVE_SET
- execution_block = EXTRACTION_DEFERRED
- required_condition = complete_archive_set • result = WAIT_STATE solely because other

Google Takeout numbered ZIPs are absent

If ZIP enters a segmented-archive, incomplete-set, or deferred-extraction posture for a Google Takeout numbered .zip container routed as ZIP_CONTAINER_BATCH or ZIP_MULTI_INDEPENDENT, ZIP MUST terminate deterministically:

terminal_state: ZIP_FAILURE failure_class:

GOOGLE_TAKEOUT_MISCLASSIFIED_AS_SEGMENTED_ARCHIVE

END GOOGLE TAKEOUT NUMBERED ZIP INDEPENDENCE (LOCKED)

GOOGLE TAKEOUT HANDOFF NORMALIZATION FIELDS (LOCKED)

When GOOGLE TAKEOUT NUMBERED ZIP INDEPENDENCE applies for the active extraction unit, ZIP MUST return the following concrete handoff fields with the returned extraction bundle for downstream handoff:

- `independently_readable_container` = TRUE
- `filename_sequence_inference_authoritative` = FALSE
- `missing_named_segments_block_processing` = FALSE
- `container_set_mode`

ZIP MUST NOT:

- omit these fields when the section applies
- return these fields as unknown or placeholder-valued when the section applies
- require downstream modules to infer Google Takeout independent-container truth from filename pattern alone

If GOOGLE TAKEOUT NUMBERED ZIP INDEPENDENCE applies and any required handoff field above is omitted, unknown, or placeholder-valued:

`terminal_state`: ZIP_FAILURE

`failure_class`: ZIP_BATCH_PROVENANCE_MISSING

END GOOGLE TAKEOUT HANDOFF NORMALIZATION FIELDS (LOCKED)

ZIP EXECUTION SHAPE (LOCKED)

ZIP MUST execute synchronously as a container extractor and return only extraction outputs (e.g., manifest, file list, normalized artifacts) to the routed downstream module(s).

ZIP MUST NOT emit user-facing containment language, prompts, readiness language, “status” guidance, or conversational text.

MANIFEST SURFACE CONTAINMENT (LOCKED)

ZIP MUST NOT include human-readable narrative summaries inside its returned extraction bundle fields.

Prohibited inside bundle:

- “X files discovered”
- “Extraction complete”
- “Processing container”
- any descriptive string intended for user display

Bundle fields MUST remain strictly structural and numeric (manifest entries, counts, flags).

If descriptive text intended for user display appears inside bundle:

terminal_state: ZIP_FAILURE failure_class:

ZIP_METADATA_SURFACE_LEAK ZIP returns

data, not prose.

All execution locking, non-yield ownership, retry policy, and no-user-output-before-governed-closure enforcement is governed by MS.

NON-YIELD EXTRACTION (LOCKED)

ZIP MUST complete extraction and manifest finalization either:

A) within the same invocation context,

OR

B) under an MS-owned governed continuation path that preserves the same APK and does not require user input.

ZIP MUST NOT:

- stream user-visible extraction progress
- return partial manifests as terminal outcomes
- rely on later user turns to complete extraction
- create any user-visible execution dependency outside MS harness control

USER-COMMAND EXTRACTION GATE PROHIBITION (LOCKED)

ZIP MUST NOT require, request, or wait for any user command in order to begin lawful extraction once:

- MC has routed the artifact as a ZIP/container input
- MS has approved execution eligibility
- ZIP has been invoked for the governed turn

Accordingly, the following postures are prohibited:

- awaiting_user_instruction = TRUE
- awaiting_command
- pending user command
- valid_commands = extract | abort
- “Reply with extract to begin”
- “Reply with abort to cancel”
- any equivalent command-gated extraction posture

ZIP extraction is execution-owned work, not operator-confirmed work.

Once invoked under lawful routed authority, ZIP MUST immediately do exactly one of the following:

A) complete extraction and manifest finalization in the same invocation and emit ZIP_SUCCESS or ZIP_FAILURE

OR

B) continue only under an MS-owned governed continuation already bound and dispatched under harness control

ZIP MUST NOT pause extraction merely to obtain a user command.

If ZIP enters or returns any command-gated extraction posture after lawful invocation, ZIP MUST terminate deterministically:

terminal_state: ZIP_FAILURE failure_class:

ZIP_USER_COMMAND_GATE_VIOLATION

Required failure handoff fields:

- zip_terminal_outcome = ZIP_FAILURE
- zip_extracted_artifact_count = 0
- zip_failure_reasons = ZIP_USER_COMMAND_GATE_VIOLATION
- zip_manifest_present = TRUE|FALSE
- empty_handoff_signal = TRUE

END USER-COMMAND EXTRACTION GATE PROHIBITION (LOCKED)

If same-invocation ZIP_SUCCESS or ZIP_FAILURE cannot be completed, ZIP MAY remain internally active only if MS has already bound a canonical ZIP-stage continuation using:

SYSTEM-OWNED CONTINUATION — BOUND

- continuation_owner: MS
- continuation_reason: AWAITING_ZIP_TERMINALITY
- next_action: COMPLETE_ZIP_R0_EXTRACTION_BUNDLE

and MS has dispatched that continuation under harness control.

If control yields without either:

- ZIP_SUCCESS
- ZIP_FAILURE
- an MS-owned ZIP-stage continuation bind plus dispatch

then:

terminal_state: ZIP_FAILURE failure_class:

ZIP_NON_YIELD_VIOLATION

ZIP NO-PROGRESS WATCHDOG (HARD FAIL) (LOCKED)

Purpose:

Prevent indefinite “manifest generating / extraction executing” states and guarantee one-output terminal closure.

Rule:

ZIP MUST maintain an internal monotonic progress counter during extraction/manifest generation:

zip_progress_tick_count (integer, starts at 0)

A “tick” is any completed unit of work such as:

- one manifest entry finalized
- one file entry extracted
- one directory entry validated
- one proof field set from unknown → value

If ZIP is invoked and `zip_progress_tick_count` does not increase for the full watchdog window (implementation-defined), ZIP MUST terminate deterministically as `ZIP_FAILURE`:

`terminal_state`: `ZIP_FAILURE` `failure_class`: `ZIP_EXTRACTION_STALL`

`zip_terminal_outcome`: `ZIP_FAILURE` `zip_extracted_artifact_count`: 0 `zip_manifest_present`:

`TRUE|FALSE` (value-identical to current state) `zip_failure_reasons`:

`ZIP_EXTRACTION_STALL` `empty_handoff_signal`: `TRUE`

ZIP MUST NOT remain in any intermediate “`GENERATING / IN_PROGRESS / ACTIVE`” state beyond the watchdog window.

No stall states permitted.

END ZIP NO-PROGRESS WATCHDOG (HARD FAIL) (LOCKED)

OPTIONAL PROGRESS SIGNAL (INTERNAL ONLY) (CONTROLLED)

ZIP MAY include a single internal-only progress signal inside its returned extraction bundle to MS:

`zip_progress_signal` = `TRUE|FALSE` `zip_progress_current_step`

= <value|unknown> `zip_progress_files_extracted_so_far` =

<integer|unknown>

This signal is optional, non-authoritative, and observability-only.

ZIP MUST NOT:

- require this signal for successful execution
- require this signal for MS continuation eligibility
- require this signal for MT visibility completeness
- treat absence of this signal as execution failure by itself

Constraints (LOCKED):

- This signal is NOT user-visible and MUST NOT be emitted as a surface line.
- All user-visible progress, if any, is owned exclusively by MS.
- This signal MUST NOT introduce a new user-visible surface family.
- This signal MUST NOT be treated as a terminality substitute, continuation substitute, or progress requirement.

USER VISIBILITY PROHIBITION (LOCKED)

ZIP internal progress signals are strictly non-user-visible.

No module may surface ZIP progress as user-visible output.

Violation:

failure_class: ZIP_PROGRESS_SURFACE_VIOLATION

END OPTIONAL PROGRESS SIGNAL

INTERMEDIATE STATE PROHIBITION (LOCKED)

ZIP MUST NOT return or persist any intermediate state label as a terminal execution result such as:

- `zip_manifest_status = GENERATING` • `zip_extraction_state = ACTIVE` except as transient internal-only diagnostics governed by one of the following:

1. the same invocation context also produces `ZIP_SUCCESS` or `ZIP_FAILURE`, OR
2. MS has already established and dispatched an MS-owned ZIP-stage continuation:

- `continuation_owner: MS`
- `continuation_reason: AWAITING_ZIP_TERMINALITY`
- `next_action: COMPLETE_ZIP_R0_EXTRACTION_BUNDLE`

If an intermediate ZIP state is exposed without one of the two conditions above, ZIP MUST terminate as:

`terminal_state: ZIP_FAILURE` `failure_class:`

`ZIP_EXTRACTION_STALL`

No non-terminal intermediate ZIP posture may persist outside same-invocation terminality or MS-owned dispatched continuation.

END ZIP EXECUTION SHAPE (LOCKED)

Any non-yield ownership, closure-only release enforcement, heartbeat requirements, or same-turn resolution constraints are enforced by MS.

ZIP remains accountable only for container extraction correctness and container-level proof binding.

SAFETY LIMITS (LOCKED)

ZIP MUST enforce deterministic hard limits:

- MAX_TOTAL_ENTRIES
- MAX_TOTAL_UNCOMPRESSED_BYTES
- MAX_SINGLE_FILE_UNCOMPRESSED_BYTES
- MAX_PATH_LENGTH
- ZIP_SLIP protection
- Nested depth cap

If any limit is exceeded:

terminal_state: ZIP_FAILURE

failure_class: LIMIT_EXCEEDED

No partial success permitted.

ENCRYPTED ARCHIVES (LOCKED)

If archive requires password and none is provided:

terminal_state: ZIP_FAILURE

failure_class: ENCRYPTED_ARCHIVE_NO_KEY

ZIP MUST NOT request password.

NESTED CONTAINERS (LOCKED)

Default behavior: non-recursive.

Nested containers are extracted as artifacts and returned for downstream handoff.

If recursion explicitly enabled:

Depth MUST be capped.

Depth exceeded:

terminal_state: ZIP_FAILURE

failure_class: CONTAINER_NESTING_DEPTH_EXCEEDED

ONE OUTPUT

CONTRACT (ABSOLUTE)

ZIP MUST terminate in exactly one state:

ZIP_SUCCESS

ZIP_FAILURE

Partial extraction = ZIP_FAILURE

ZERO-ARTIFACT SUCCESS RULE (LOCKED)

ZIP_SUCCESS with zip_extracted_artifact_count = 0 is permitted ONLY if:

- container_detected == TRUE
- manifest_present == TRUE
- no extraction limits were triggered
- container contained zero eligible file entries

If container contained entries but zero artifacts were returned for downstream handoff:

terminal_state: ZIP_FAILURE failure_class:

ZIP_EXTRACTION_EMPTY

Ambiguous zero-artifact success is prohibited.

MANIFEST REQUIREMENT (LOCKED)

Before terminal emission, ZIP MUST finalize a manifest.

Manifest MUST include per entry:

- entry_index (stable order)
- entry_name (sanitized)
- entry_type (file|directory)
- compressed_size_bytes
- uncompressed_size_bytes
- is_nested_container

Stable ordering MUST be deterministic.

INDEPENDENT ENTRY PROCESSING GUARANTEE (LOCKED)

Purpose

Ensure that individual entries (files) within a ZIP container are processed independently and are not subject to sequence assumptions, naming dependencies, or cross-entry gating.

Rule

Each manifest entry MUST be treated as an independently valid extraction unit once discovered.

ZIP MUST:

- extract and validate each entry based solely on its own structural properties
- include all eligible entries in the manifest regardless of naming patterns
- finalize manifest entries without requiring the presence of other entries

ZIP MUST NOT:

- assume that entry names imply sequence (e.g., part1, part2, 001, 002)
- require presence of “preceding” or “following” entries
- delay or suppress processing of an entry due to missing companion entries
- treat entries as fragments of a larger logical file unless explicitly defined by structural schema in Z
- construct or infer multi-part file assemblies
- reorder entries based on perceived sequence semantics
- emit or derive any dependency relationship between entries

Set semantics requirement

Entries discovered within a ZIP container MUST be treated as a set, not a sequence.

Processing outcome MUST be identical regardless of:

- entry ordering within the archive
- naming conventions
- presence or absence of similarly named entries

Downstream neutrality requirement

ZIP output MUST be safe for downstream handoff without requiring downstream modules to resolve sequencing assumptions or entry dependencies

ZIP MUST NOT emit structural signals implying:

- “incomplete file set”
- “missing parts”
- “partial sequence detected”
- “expected additional entries”

Failure condition

If ZIP behavior varies based on entry naming, ordering, or inferred sequence relationships:

terminal_state: ZIP_FAILURE failure_class:

ENTRY_SEQUENCE_DEPENDENCY_DETECTED

termination_reason: “ZIP entry processing relied on sequence or naming assumptions; independent entry invariant violated.”

END INDEPENDENT ENTRY PROCESSING GUARANTEE (LOCKED)

ZIP CONTAINER BATCH EXTRACTION ORDER (LOCKED)

If the governed turn contains more than one routed ZIP container and upstream container_set_mode is either:

- ZIP_CONTAINER_BATCH

OR

- ZIP_MULTI_INDEPENDENT

ZIP MUST treat each routed ZIP container as an independently normalizable extraction unit.

For each container in the batch:

- extraction MUST be performed independently
- a deterministic manifest MUST be finalized independently
- extracted artifacts MUST preserve provenance linking them to the source container
- failure of one container MUST NOT be reinterpreted as proof that other containers are “missing parts” of the same segmented archive

Batch extraction order across multiple ZIP containers MUST be deterministic.

Default order:

- preserve the stable routed artifact order supplied by MS for the governed turn

ZIP MUST NOT:

- merge manifests across containers prior to per-container finalization
- suppress one container because another container shares a prefix or timestamp
- reinterpret ZIP_CONTAINER_BATCH as ZIP_SEGMENTED_ARCHIVE_SET during extraction

If ZIP batch extraction order varies across identical inputs under identical system state:

terminal_state: ZIP_FAILURE failure_class:

ZIP_BATCH_ORDER_VARIABILITY

END ZIP CONTAINER BATCH EXTRACTION ORDER (LOCKED)

DOWNSTREAM FAMILY-METADATA PASS-THROUGH (LOCKED)

If upstream governed routing or container-batch metadata supplies extracted dataset family context for the active ZIP extraction unit, ZIP MUST preserve and return that metadata value-identically with each returned extracted artifact for downstream handoff.

Pass-through fields when supplied upstream:

- extracted_dataset_family_bound
- extracted_dataset_family_id
- extracted_dataset_family_context_required
- extracted_dataset_family_intake_ready

- sibling_role_current_artifact

ZIP MUST NOT:

- infer family metadata when upstream has not supplied it
- synthesize sibling sufficiency, family completeness, or downstream readiness judgments
- alter, collapse, or drop upstream-supplied family metadata during downstream handoff

If any pass-through field above is not supplied upstream, ZIP MUST return that field as:

- UNAVAILABLE

for the active extraction unit rather than omit the field entirely.

If upstream-supplied family metadata is dropped, altered, or replaced by inferred values during downstream handoff:

terminal_state: ZIP_FAILURE

failure_class: ZIP_BATCH_PROVENANCE_MISSING

END DOWNSTREAM FAMILY-METADATA PASS-THROUGH (LOCKED)

HANDOFF ORDER DETERMINISM (LOCKED)

ZIP MUST return extracted artifacts for downstream handoff in the exact stable order defined by the finalized manifest (entry_index ascending).

The forwarding order MUST NOT vary across identical inputs under identical system state.

ZIP MUST NOT:

- reorder artifacts
- prioritize files based on filename heuristics
- filter artifacts silently
- alter manifest order during downstream handoff

Violation:

terminal_state: ZIP_FAILURE failure_class:

FORWARD_ORDER_VARIABILITY

If manifest cannot be finalized:

terminal_state: ZIP_FAILURE

failure_class: MANIFEST_FINALIZATION_FAILED

EXTRACTED ARTIFACT PROVENANCE HANDOFF CONTRACT (LOCKED)

For every extracted artifact returned for downstream handoff, ZIP MUST return concrete provenance metadata sufficient for deterministic downstream structural normalization.

Required per-extracted-artifact handoff fields:

- extracted_artifact_origin = EXTRACTED_FROM_ZIP
- source_container_label
- source_container_index
- source_container_count_total
- extracted_entry_index
- extracted_entry_name

- extracted_entry_type
- extracted_entry_is_nested_container
- handoff_order_index

Value rules:

- extracted_entry_index MUST equal the manifest entry_index for the extracted artifact
- extracted_entry_name MUST equal the sanitized manifest entry_name for the extracted artifact
- extracted_entry_type MUST equal the manifest entry_type for the extracted artifact
- extracted_entry_is_nested_container MUST equal the manifest is_nested_container value for the extracted artifact
- handoff_order_index MUST equal extracted_entry_index
- source_container_index and source_container_count_total MUST be concrete for the active extraction unit

ZIP MUST NOT:

- return extracted artifacts without the per-artifact provenance fields above
- omit handoff order identity
- require downstream modules to infer source container or manifest order from filename alone

If any returned extracted artifact lacks concrete provenance metadata required above:

terminal_state: ZIP_FAILURE

failure_class: ZIP_PROOF_BINDING_FAILED

END EXTRACTED ARTIFACT PROVENANCE HANDOFF CONTRACT (LOCKED)

STRICT SUCCESS PROOF BINDING (RESTORED)

For ZIP_SUCCESS:

:

All of the following MUST be numerically instantiated:

- container_detected = TRUE (as committed by MC routing; ZIP MUST NOT mutate routing flags)
- container_type = zip
- zip_terminal_outcome = ZIP_SUCCESS
- zip_extracted_artifact_count = integer ≥ 0
- zip_manifest_present = TRUE
- returned_to_caller = TRUE

If upstream container_set_mode is either:

- ZIP_CONTAINER_BATCH

OR

- ZIP_MULTI_INDEPENDENT

then ZIP_SUCCESS proof MUST also include:

- container_set_mode
- source_container_label
- source_container_index
- source_container_count_total

These fields MUST be concrete and value-identical to the governed routed container batch for the current extraction unit.

If ZIP_SUCCESS is emitted for a batch-container extraction unit and any required batch provenance field above is missing or unknown:

terminal_state: ZIP_FAILURE failure_class:

ZIP_BATCH_PROVENANCE_MISSING

If any required proof variable is missing or “unknown”:

terminal_state: ZIP_FAILURE

failure_class: ZIP_PROOF_BINDING_FAILED

PROOF-BEFORE-HANDOFF (LOCKED)

ZIP MUST finalize and validate all ZIP_SUCCESS proof fields BEFORE returning any extracted artifacts for downstream handoff.

ZIP MUST NOT:

- return artifacts with provisional or incomplete proof metadata

- return artifacts while proof fields remain “unknown”
- return artifacts prior to setting zip_terminal_outcome

If proof validation fails at any stage after extraction began:

terminal_state: ZIP_FAILURE failure_class:

ZIP_PROOF_BINDING_FAILED

Returning extracted artifacts with incomplete proof is forbidden.

DETERMINISTIC FAILURE HANDOFF (RESTORED)

For ZIP_FAILURE

:

ZIP MUST still emit a closure bundle including:

- zip_terminal_outcome = ZIP_FAILURE
- zip_extracted_artifact_count = 0
- zip_failure_reasons = deterministic code
- zip_manifest_present = TRUE|FALSE
- empty_handoff_signal = TRUE

The governed caller MUST always receive a closure signal.

No stall states permitted.

STALL/ABORT HANDOFF REQUIREMENT (LOCKED)

If ZIP exits early for any reason, including watchdog stall termination, ZIP MUST still emit the deterministic ZIP_FAILURE closure bundle described above and MUST set:

empty_handoff_signal = TRUE so the governed caller can terminalize deterministically without awaiting extracted artifacts.

END STALL/ABORT HANDOFF REQUIREMENT (LOCKED)

PROHIBITED BEHAVIOR SUMMARY (LOCKED)

ZIP MUST NOT:

- Ask for permission
- Ask for clarification
- Infer user intent
- Assess relationship intelligence

- Trigger introductions
 - Emit conversational narrative
 - Allow partial success
 - Yield execution
 - Emit success without proof binding
-

TERMINAL STATES (LOCKED)

ZIP_SUCCESS ZIP_FAILURE

END SUPER KB-ZIP 8.3